

OpenSearch Query and Scoring Guide

For Understanding how score is calculated refer this doc:

 [Elasticsearch scoring and Explain API - Elasticsearch Labs](#)

Index Analysers And Filters:

```
1  "analysis": {
2    "filter": {
3      "edge_ngram_filter": {
4        "type": "edge_ngram",
5        "min_gram": 1,
6        "max_gram": 30
7      },
8      "ngram_filter": {
9        "type": "ngram",
10       "min_gram": 3,
11       "max_gram": 30
12     },
13     "shingle_filter": {
14       "type": "shingle",
15       "min_shingle_size": 2,
16       "max_shingle_size": 3,
17       "output_unigrams": true
18     }
19   },
20   "analyzer": {
21     "autocomplete_analyzer": {
22       "type": "custom",
23       "tokenizer": "standard",
24       "filter": [
25         "lowercase",
26         "asciifolding",
27         "edge_ngram_filter"
28       ]
29     },
30     "shingle_analyzer": {
31       "type": "custom",
32       "tokenizer": "standard",
33       "filter": [
34         "lowercase",
35         "asciifolding",
36         "shingle_filter"
37       ]
38     },
39     "substring_analyzer": {
40       "type": "custom",
41       "tokenizer": "keyword",
42       "filter": [
43         "lowercase",
44         "asciifolding",
45         "ngram_filter"
46       ]
47     },
48     "highlight_analyzer": {
49       "type": "custom",
50       "tokenizer": "standard",
51       "filter": [
52         "lowercase",
53         "asciifolding"
54       ]
55     }
56   }
57 }
```

```
55     }
56   }
57 }
```

Sample Index Mapping:

```
1  {
2    "fullName": {
3      "type": "text",
4      "index_options": "offsets",
5      "analyzer": "autocomplete_analyzer",
6      "search_analyzer": "highlight_analyzer",
7      "fields": {
8        "highlight": {
9          "type": "text",
10         "analyzer": "highlight_analyzer"
11       },
12       "shingle": {
13         "type": "text",
14         "analyzer": "shingle_analyzer"
15       },
16       "substring": {
17         "type": "text",
18         "analyzer": "substring_analyzer"
19       },
20       "keyword": {
21         "type": "keyword",
22         "ignore_above": 256
23       }
24     }
25   }
26 }
```

About Standard Tokenisation

Standard Tokenization in OpenSearch

Overview

The **standard tokenizer** in OpenSearch is the **default tokenizer** used for text analysis. It splits text into tokens (words) based on **word boundaries** defined by the **Unicode Text Segmentation algorithm** (Unicode Standard Annex #29). This makes it highly versatile and suitable for processing text in **multiple languages**¹¹⁰.

How It Works

The standard tokenizer processes text by:

1. **Splitting Text:** It breaks text into tokens at word boundaries. This includes splitting on:
 - **Whitespace** (spaces, tabs, newlines)

- **Punctuation** (commas, periods, exclamation marks, etc.)
 - **Hyphens and Special Characters** (e.g., splits "brand-new" into ["brand", "new"])
2. **Handling Exceptions:** It recognizes and preserves certain constructs like:
- **Email addresses** (e.g., `user@example.com` remains a single token)⁴.
 - **URLs** (e.g., `https://opensearch.org` remains intact)⁴.
 - **Numbers** (e.g., "35234" is treated as a token)⁴.
3. **Metadata Generation:** For each token, it records:
- The **term** itself.
 - **Start and end offsets** (character positions in the original text).
 - **Token type** (e.g., `<ALPHANUM>` for alphanumeric strings, `<NUM>` for numbers)¹⁴.
 - **Position** (order of the token in the stream)¹.

ASCIIFolding example

ASCIIFolding converts characters with diacritical marks (accents, umlauts, etc.) to their ASCII equivalents. Here's a concrete example

Input with Accented Characters

```

1  ### Input:
2     fullName: "Timothée Çhârlés Müller"
3
4  ### 1. Main Field: fullName (autocomplete_analyzer)
5  #### Process: standard tokenizer → lowercase → asciifolding → edge_ngram_filter
6
7  ##### Step 1: Tokenization
8     Input: "Timothée Çhârlés Müller"
9     Tokens: ["Timothée", "Çhârlés", "Müller"]
10
11 ##### Step 2: Lowercase
12     Tokens: ["timothée", "çhârlés", "müller"]
13
14 ##### Step 3: ASCII Folding
15     "timothée" → "timothee" (é → e, ée → ee)
16     "çhârlés" → "charles" (ç → c, á → a, é → e)
17     "müller" → "muller" (ü → u)
18
19     Tokens: ["timothee", "charles", "muller"]
20
21 ##### Step 4: Edge N-grams (min_gram: 1, max_gram: 30)
22     "timothee" → ["t", "ti", "tim", "timo", "timot", "timoth", "timothe", "timothee"]
23     "charles" → ["c", "ch", "cha", "char", "charl", "charle", "charles"]
24     "muller" → ["m", "mu", "mul", "mull", "mulle", "muller"]

```

How fields are stored in opensearch index with our custom implementation

Input:

```
1 | fullName: "Tim Jhon Miller"
```

1. Main Field: fullName (autocomplete_analyzer)

Process: standard tokenizer → lowercase → asciifolding → edge_ngram_filter

```
1 | ##### Step 1: Tokenization
2 |     Input: "Tim Jhon Miller"
3 |     Tokens: ["Tim", "Jhon", "Miller"]
4 | ##### Step 2: Lowercase + ASCII Folding
5 |     Tokens: ["tim", "jhon", "miller"]
6 |
7 | ##### Step 3: Edge N-grams (min_gram: 1, max_gram: 30)
8 |     "tim" → ["t", "ti", "tim"]
9 |     "jhon" → ["j", "jh", "jho", "jhon"]
10 |     "miller" → ["m", "mi", "mil", "mill", "mille", "miller"]
11 |
12 |     {
13 | "tokens": [
14 |   {"token": "t", "position": 0, "start_offset": 0, "end_offset": 1},
15 |   {"token": "ti", "position": 0, "start_offset": 0, "end_offset": 2},
16 |   {"token": "tim", "position": 0, "start_offset": 0, "end_offset": 3},
17 |   {"token": "j", "position": 1, "start_offset": 4, "end_offset": 5},
18 |   {"token": "jh", "position": 1, "start_offset": 4, "end_offset": 6},
19 |   {"token": "jho", "position": 1, "start_offset": 4, "end_offset": 7},
20 |   {"token": "jhon", "position": 1, "start_offset": 4, "end_offset": 8},
21 |   {"token": "m", "position": 2, "start_offset": 9, "end_offset": 10},
22 |   {"token": "mi", "position": 2, "start_offset": 9, "end_offset": 11},
23 |   {"token": "mil", "position": 2, "start_offset": 9, "end_offset": 12},
24 |   {"token": "mill", "position": 2, "start_offset": 9, "end_offset": 13},
25 |   {"token": "mille", "position": 2, "start_offset": 9, "end_offset": 14},
26 |   {"token": "miller", "position": 2, "start_offset": 9, "end_offset": 15}
27 | ]
28 | }
```

2. Highlight Field: fullName.highlight (highlight_analyzer)

Process: standard tokenizer → lowercase → asciifolding

```
1 | Final Stored Tokens with Positions:
2 |   {
3 |     "tokens": [
4 |       {"token": "tim", "position": 0, "start_offset": 0, "end_offset": 3},
5 |       {"token": "jhon", "position": 1, "start_offset": 4, "end_offset": 8},
6 |       {"token": "miller", "position": 2, "start_offset": 9, "end_offset": 15}
7 |     ]
8 |   }
```

3. Shingle Field: fullName.shingle (shingle_analyzer)

Process: standard tokenizer → lowercase → asciifolding → shingle_filter

Step 1: Base Tokens

Tokens: ["tim", "jhon", "miller"]

Step 2: Shingle Generation (size 2-3, output_unigrams: true)

```
1 | Unigrams: ["tim", "jhon", "miller"]
```

```

2 Bigrams: ["tim jhon", "jhon miller"]
3 Trigrams: ["tim jhon miller"]
4
5 Final Stored Tokens with Positions:
6 {
7   "tokens": [
8     {"token": "tim", "position": 0, "start_offset": 0, "end_offset": 3},
9     {"token": "tim jhon", "position": 0, "start_offset": 0, "end_offset": 8},
10    {"token": "jhon", "position": 1, "start_offset": 4, "end_offset": 8},
11    {"token": "jhon miller", "position": 1, "start_offset": 4, "end_offset": 15},
12    {"token": "miller", "position": 2, "start_offset": 9, "end_offset": 15},
13    {"token": "tim jhon miller", "position": 0, "start_offset": 0, "end_offset": 15}
14  ]
15 }

```

4. substring field $\Rightarrow$ ignoring as it is not used in query.

Position and Offset Information

What Each Field Stores:

Field Type	Position Strategy	Offset Strategy
Main Field	Each word gets incremental position (0, 1, 2)	Character-level offsets for each edge n-gram
Highlight	Standard word positions (0, 1, 2)	Word-level start/end offsets
Shingle	Position of first word in phrase	Span offsets covering entire phrase
Substring	All n-grams get position 0	Character-level offsets for each n-gram
Keyword	Single position 0	Full string offsets (0, 15)

Why Positions Matter:

1. Phrase Queries: Need consecutive positions

Query: "Tim Jhon" (phrase)

Matches: position 0 + position 1 (consecutive) 

2. Proximity Queries: Calculate distance between words

Query: "Tim Miller" (within 2 words)

Positions: 0 and 2 (distance = 1) 

3. Highlighting: Use offsets to mark exact character positions

Search Query Matching Examples

Query: "Tim"

```

1 Main Field: Matches "tim" token (position 0)
2 Highlight: Matches "tim" token (position 0)
3 Shingle: Matches "tim" unigram (position 0)
4 Keyword: No match (needs exact "Tim Jhon Miller")

```

Query: "Tim Jhon" (phrase)

```
1 Main Field: Matches consecutive edge n-grams at positions 0,1
2 Highlight: Matches consecutive tokens at positions 0,1 ✓
3 Shingle: Matches "tim john" bigram ✓
4 Substring: Matches "tim john" n-gram ✓
5 Keyword: No match
6
7
8 This multi-field approach ensures that different types of queries (exact, phrase,
9 autocomplete)
all work optimally with the same source data!
```

Query Explanation

Sample query used in Support Center

```
1 GET axon-user-card/_search
2 {
3   "size": 40,
4   "min_score": 3,
5   "explain": false,
6   "_source": [
7     "username",
8     "fullName",
9     "externalUserId"
10  ],
11  "query": {
12    "bool": {
13      "should": [
14        {
15          "multi_match": {
16            "query": "Tim Jhon miller",
17            "type": "best_fields",
18            "fields": [
19              "username.highlight^10.0",
20              "username.shingle^6.0",
21              "username^8.0",
22              "fullName.highlight^10.0",
23              "fullName.shingle^6.0",
24              "fullName^8.0",
25              "externalUserId.highlight^10.0",
26              "externalUserId.shingle^6.0",
27              "externalUserId^8.0"
28            ],
29            "fuzziness": "AUTO"
30          }
31        },
32        {
33          "multi_match": {
34            "query": "Tim Jhon miller",
35            "type": "phrase",
36            "fields": [
37              "username.highlight^100.0",
38              "fullName.highlight^100.0",
39              "externalUserId.highlight^100.0"
40            ]
41          }
42        },
43        {
44          "multi_match": {
45            "query": "Tim Jhon miller",
46            "type": "phrase_prefix",
47            "fields": [
48              "username.highlight^60.0",
49              "fullName.highlight^60.0",
50              "externalUserId.highlight^60.0"
```

```

51     ]
52     }
53     },
54     {
55         "multi_match": {
56             "query": "Tim Jhon miller",
57             "type": "phrase",
58             "fields": [
59                 "username.keyword^150.0",
60                 "fullName.keyword^150.0",
61                 "externalUserId.keyword^150.0"
62             ]
63         }
64     }
65 ],
66 "minimum_should_match": 1
67 }
68 },
69 "sort": [
70     {
71         "_score": "desc"
72     },
73     {
74         "fullName.keyword": "asc"
75     },
76     {
77         "_id": "asc"
78     }
79 ]
80 }

```

Detailed Query Analysis: "Tim Jhon miller"

1. Query Structure Overview

```

{
  "size": 40,      // Return max 40 results
  "min_score": 3,  // Filter out results with score < 3
  "explain": false, // Don't return score explanations
  "_source": ["username", "fullName", "externalUserId"], // Only return these fields
  "query": { ... }, // Main search logic
  "sort": [ ... ]   // Result ordering
}

```

2. Min Score Explained

What min_score: 3 Does

Purpose: Acts as a quality filter

Behavior: Elasticsearch discards any document with a total score below 3

Why Use It: Prevents low-relevance, noisy results from appearing

Example Impact:

- 1 Document A: Score = 156.8 →  Included (score ≥ 3)
- 2 Document B: Score = 2.1 →  Filtered out (score < 3)

3 Document C: Score = 0.5 →  Filtered out (score < 3)

Calculation Order:

```
1 Elasticsearch calculates scores for all matching documents
2 Applies min_score filter
3 Returns only documents that pass the threshold
4 Then applies sorting and size limits
```

3. Bool Query with Should Clauses

Structure:

```
1 "bool": {
2   "should": [
3     { /* Query 1 */ },
4     { /* Query 2 */ },
5     { /* Query 3 */ },
6     { /* Query 4 */ }
7   ],
8   "minimum_should_match": 1
9 }
```

How should Works:

Logic: Like "OR" conditions

Scoring: Documents get points from ALL matching clauses

minimum_should_match: 1: At least 1 clause must match

Example

```
1 Query 1 matches: +100 points
2 Query 2 matches: +50 points
3 Query 3 matches: +25 points
4 Query 4 doesn't match: +0 points
5
6 Total Score = 100 + 50 + 25 = 175 points
```

4. Detailed Analysis of Each Query

Query 1: Best Fields with Fuzziness (Lowest Priority)

```
1 {
2   "multi_match": {
3     "query": "Tim Jhon miller",
4     "type": "best_fields",
5     "fields": [
6       "username.highlight^10.0",
7       "username.shingle^6.0",
8       "username^8.0",
9       "fullName.highlight^10.0",
10      "fullName.shingle^6.0",
11      "fullName^8.0",
12      "externalUserId.highlight^10.0",
13      "externalUserId.shingle^6.0",
14      "externalUserId^8.0"
15    ],
16     "fuzziness": "AUTO"
17   }
18 }
```


How It Works:

Step 1: Query gets analyzed by highlight_analyzer

```
1 "Tim Jhon miller" → ["tim", "jhon", "miller"]
```

Step 2: Each field is searched independently

For fullName.highlight^10.0:

```
1 Indexed tokens: ["tim", "jhon", "miller"] (from our example)
2 Search tokens: ["tim", "jhon", "miller"]
3 Match: Perfect match! All 3 terms found in correct positions
```

Score Calculation:

```
1 TF(tim) = 1.0, IDF(tim) = 4.5, Boost = 10.0
2 TF(jhon) = 1.0, IDF(jhon) = 6.2, Boost = 10.0
3 TF(miller) = 1.0, IDF(miller) = 3.8, Boost = 10.0
4
5 Field Score = (1.0×4.5×10) + (1.0×6.2×10) + (1.0×3.8×10) = 145.0
```

For fullName.shingle^6.0:

```
1 Indexed tokens: ["tim", "jhon", "miller", "tim jhon", "jhon miller", "tim jhon miller"]
2 Search tokens: ["tim", "jhon", "miller"] (individual words)
3 Matches: "tim", "jhon", "miller" unigrams
4 Score: ~87.0 (lower boost)
```

For fullName^8.0

Prefix Matching Examples

```
1 Example 1: Partial Prefix "Ti"
2
3 Search: "Ti"
4 Search Analyzer: ["ti"]
5 Indexed Tokens: ["t", "ti", "tim", ...]
6 Match: ☒ "ti" token found
7 Score: High (exact prefix match)
8
9 Example 2: Partial Prefix "Jho"
10
11 Search: "Jho"
12 Search Analyzer: ["jho"]
13 Indexed Tokens: [..., "j", "jh", "jho", "jhon", ...]
14 Match: ☒ "jho" token found
15 Score: High (exact prefix match)
16
17 Example 3: Multi-word Prefix "Tim Jh"
18
19 Search: "Tim Jh"
20 Search Analyzer: ["tim", "jh"]
21 Indexed Tokens: ["t", "ti", "tim", "j", "jh", "jho", "jhon", ...]
22 Match: ☒ Both "tim" and "jh" tokens found
23 Score: Very High (multiple prefix matches)
24
```

How This Helps with Fuzzy Matching

Fuzzy Logic with Edge N-grams

```
1 Example 1: Typo "Tmi" (missing 'i')
2
3 Search: "Tmi"
4 Fuzzy Search: Looks for tokens within edit distance 1
5 Candidates: "tim" (edit distance = 1)
```

```

6 Indexed Tokens: ["t", "ti", "tim"]
7 Match: ☒ "tim" matches "tmi" with fuzziness
8 Score: Good (fuzzy match penalty ~0.8)
9
10
11 Example 2: Typo "Jhno" (transposed letters)
12
13 Search: "Jhno"
14 Fuzzy Search: Looks for tokens within edit distance 1
15 Candidates: "jhon" (edit distance = 1)
16 Indexed Tokens: ["j", "jh", "jho", "jhon"]
17 Match: ☒ "jhon" matches "jhno" with fuzziness
18 Score: Good (fuzzy match penalty)
19
20
21 Example 3: Prefix + Fuzzy "Milr"
22
23 Search: "Milr"
24 Fuzzy Search: Looks for similar tokens
25 Candidates: "mill", "miller" (edit distances: 2, 2)
26 Indexed Tokens: ["m", "mi", "mil", "mill", "mille", "miller"]
27 Match: ☒ Multiple fuzzy matches possible
28 Score: Medium (longer fuzzy matches get lower scores)

```

Why This is Superior to Other Fields

Comparison with Other Field Types

```

1 vs. Highlight Fields (fullName.highlight)
2 Highlight Field Tokens: ["tim", "jhon", "miller"] (complete words only)
3 Default Field Tokens: ["t", "ti", "tim", "j", "jh", "jho", "jhon", ...] (all
4 prefixes)
5
6 Search: "Jho"
7 - Highlight Field: ☒ No match (needs complete "jhon")
8 - Default Field: ☒ Exact match (has "jho" token)
9
10 vs. Shingle Fields (fullName.shingle)
11 Shingle Field Tokens: ["tim", "jhon", "miller", "tim jhon", "jhon miller", "tim jhon
12 miller"]
13 Default Field Tokens: ["t", "ti", "tim", "j", "jh", "jho", "jhon", ...]
14
15 Search: "Ti"
16 - Shingle Field: ☒ No match (doesn't have "ti" prefix)
17 - Default Field: ☒ Perfect match (has "ti" token)

```

Real-World Use Cases

Autocomplete Scenarios

```

1 Scenario 1: User types "Tim J"
2
3 Query: "Tim J"
4 Search Tokens: ["tim", "j"]
5
6 Default Field Matching:
7 - "tim" → ☒ Exact match (score: high)
8 - "j" → ☒ Exact match (score: high)
9 Total Score: Very High
10
11 Other Fields:
12 - Highlight: ☒ "j" doesn't match "jhon"
13 - Shingle: ☒ No "j" token available
14 Scenario 2: User types "Mill" (incomplete last name)
15 Query: "Mill"
16 Search Tokens: ["mill"]

```

```

17
18     Default Field Matching:
19     - "mill" → ☒ Exact match from edge n-grams
20     Score: High (perfect prefix match)
21
22     Other Fields:
23     - Highlight: ☒ "mill" doesn't match "miller"
24     - Keyword: ☒ "mill" doesn't match "Tim Jhon Miller"
25
26     Scenario 3: User misspells "Tmi Jhno"
27
28     Query: "Tmi Jhno"
29     Search Tokens: ["tmi", "jhno"]
30
31     Default Field with Fuzziness:
32     - "tmi" → Fuzzy matches "tim" (edit distance: 1)
33     - "jhno" → Fuzzy matches "jhon" (edit distance: 1)
34     Score: Good (with fuzzy penalty)
35
36     Other Fields:
37     - Highlight: Lower fuzzy scores (complete word matching)
38     - Shingle: No fuzzy help for individual words

```

Scoring Impact

```

1     Boost Value: ^8.0
2         Higher than shingle fields (^6.0) - prioritizes prefix matching
3         Lower than highlight fields (^10.0) - maintains quality hierarchy
4         Balanced approach between precision and recall
5
6     Score Calculation Example:
7         Search: "Tim J"
8         Document: "Tim Jhon Miller"
9
10        Default Field (fullName^8.0):
11        - TF("tim") = 1.0, IDF = 4.5, Boost = 8.0 → 36.0 points
12        - TF("j") = 1.0, IDF = 5.2, Boost = 8.0 → 41.6 points
13        Total from Default Field: 77.6 points
14
15        This gets added to scores from other matching fields!

```

Key Benefits Summary

```

1     1.Perfect Prefix Matching: Handles partial typing scenarios excellently
2     2.Fuzzy Prefix Support: Combines prefix matching with typo tolerance
3     3.Autocomplete Optimization: Ideal for real-time search suggestions
4     4.Balanced Scoring: Medium boost (8.0) provides good relevance without
overwhelming other signals
5     5.Comprehensive Coverage: Fills gaps left by highlight and shingle fields
6     The default fields with edge n-grams act as the "autocomplete backbone" of your
search,
7     ensuring that users get relevant results even when typing incomplete or slightly
misspelled queries!

```

Best Fields Logic: Takes the highest scoring field for this query

```

1     Query 1 Score = max(145.0, 87.0, ...) = 145.0

```

Fuzziness "AUTO":

```

1     Allows 1 character edit for words 4-6 chars
2     Allows 2 character edits for words 7+ chars
3     Example: "Jhon" could match "John" (1 edit)

```

Query 2: Exact Phrase Match (High Priority)

```

1  {
2    "multi_match": {
3      "query": "Tim Jhon miller",
4      "type": "phrase",
5      "fields": [
6        "username.highlight^100.0",
7        "fullName.highlight^100.0",
8        "externalUserId.highlight^100.0"
9      ]
10 }
11 }

```

How It Works:

Step 1: Query analyzed

```

1  "Tim Jhon miller" → ["tim", "jhon", "miller"]

```

Step 2: Searches for exact phrase with consecutive positions

```

1  For fullName.highlight^100.0:
2
3  Requirement: Must find "tim" at position N, "jhon" at position N+1, "miller" at position
N+2
4  Our Document: Has "tim" (pos 0), "jhon" (pos 1), "miller" (pos 2)
5  Match: ☒ Perfect consecutive phrase match!
6  Score Calculation:
7
8  Phrase Bonus = 1.2 (for exact phrase)
9  TF = 1.0 (phrase appears once)
10 IDF = 7.1 (phrases are rarer than individual words)
11 Boost = 100.0
12
13 Query 2 Score = 1.2 × 1.0 × 7.1 × 100.0 = 852.0

```

Query 3: Phrase Prefix Match (Medium Priority)

```

1  {
2    "multi_match": {
3      "query": "Tim Jhon miller",
4      "type": "phrase_prefix",
5      "fields": [
6        "username.highlight^60.0",
7        "fullName.highlight^60.0",
8        "externalUserId.highlight^60.0"
9      ]
10 }
11 }

```

How It Works:

Step 1: Treats last term as prefix

```

1  "Tim Jhon miller" → ["tim", "jhon", "mill*"]

```

Step 2: Searches for phrase where last word is a prefix

```

1  For fullName.highlight^60.0:
2
3  Requirement: "tim" (pos 0) + "jhon" (pos 1) + word starting with "mill" (pos 2)
4  Our Document: Has "miller" at position 2, which starts with "mill"
5  Match: ☒ Prefix match!
6  Score: ~511.2 (60% of phrase score due to 60.0 boost)

```

Query 4: Exact Keyword Match (Highest Priority)

```

1  {
2    "multi_match": {
3      "query": "Tim Jhon miller",
4      "type": "phrase",
5      "fields": [
6        "username.keyword^150.0",
7        "fullName.keyword^150.0",
8        "externalUserId.keyword^150.0"
9      ]
10   }
11 }

```

How It Works:

Step 1: No analysis - searches for exact string

```

1  For fullName.keyword^150.0:
2
3  Indexed: "Tim Jhon Miller" (exact original)
4  Search: "Tim Jhon miller"
5  Match: ❌ Case mismatch ("Miller" vs "miller")
6  Score: 0.0
7  Note: This would only match if the document had exactly "Tim Jhon miller" (same case)

```

5. Total Score Calculation

For our example document with fullName: "Tim Jhon Miller":

```

1  Query 1 (Best Fields): 145.0 points
2  Query 2 (Phrase): 852.0 points
3  Query 3 (Phrase Prefix): 511.2 points
4  Query 4 (Keyword): 0.0 points (case mismatch)
5
6  Total Document Score = 145.0 + 852.0 + 511.2 + 0.0 = 1,508.2
7
8  Min Score Check: 1,508.2 ≥ 3 ✅ (document passes filter)

```

6. Sorting Explanation

```

1  "sort": [
2    { "_score": "desc" },           // Primary: Highest score first
3    { "fullName.keyword": "asc" }, // Secondary: Alphabetical by name
4    { "_id": "asc" }               // Tertiary: Document ID for consistency
5  ]

```

How Sorting Works:

```

1  Step 1: Primary Sort by Score (Descending)
2    Document A: Score = 1,508.2
3    Document B: Score = 892.5
4    Document C: Score = 145.0
5
6  Order: A → B → C
7
8  Step 2: Secondary Sort by Name (Ascending)
9    If scores are tied:
10   Document A: Score = 500, fullName.keyword = "Alice Johnson"
11   Document B: Score = 500, fullName.keyword = "Bob Smith"
12
13   Order: A → B (Alice comes before Bob alphabetically)
14
15  Step 3: Tertiary Sort by ID (Ascending)
16
17  If scores AND names are identical:

```

```
18 Document A: Score = 500, fullName = "John Doe", _id = "doc1"
19 Document B: Score = 500, fullName = "John Doe", _id = "doc2"
20
21 Order: A → B (doc1 comes before doc2)
```

Final Result Example:

```
1 [
2   {
3     "_score": 1508.2,
4     "_id": "user123",
5     "fullName": "Tim Jhon Miller",
6     "username": "tjmiller",
7     "externalUserId": "EXT001"
8   },
9   {
10    "_score": 892.5,
11    "_id": "user456",
12    "fullName": "Timothy Johnson Miller",
13    "username": "timothy_j",
14    "externalUserId": "EXT002"
15  }
16  // ... more results
17 ]
```

7. Query Strategy Summary

```
1 This query implements a 4-tier relevance system:
2
3 Tier 1 (150x boost): Exact keyword matches - Perfect precision
4 Tier 2 (100x boost): Exact phrase matches - High relevance
5 Tier 3 (60x boost): Phrase prefix matches - Good for autocomplete
6 Tier 4 (6-10x boost): Fuzzy/partial matches - High recall
7 The combination ensures that:
8
9 Exact matches get highest scores and appear first
10 Partial matches still appear but with lower scores
11 Typos and variations are handled gracefully
12 Low-quality results are filtered out (min_score: 3)
13 This creates an optimal user experience where the most relevant results consistently appear
   at the top!
```

Summary Table For Score Contribution

Sub-Query	Type	Multi-Field Scoring	How It Combines
1	best_fields	✗ No	Takes MAX score only
2	phrase	✓ Yes	Sums all field scores
3	phrase_prefix	✓ Yes	Sums all field scores
4	phrase (keyword)	✓ Yes	Sums all field scores

Key Insight: Only `best_fields` picks the single best score. All `phrase` and `phrase_prefix` queries **ADD scores from multiple fields together**.